

Towards Understanding the Value of False Positives in Static Code Analysis

Carlo Dimastrogiovanni

Polytechnic University of Turin, Italy
carlo.dimastrogiovanni@studenti.polito.it

Nuno Laranjeiro

CISUC, Department of Informatics Engineering
University of Coimbra, Portugal
cnl@dei.uc.pt

Abstract—Static code analysis is a well-known technique used to detect potential software security issues. Nowadays, given the large variety of vulnerabilities and the increasing complexity of web applications, it is difficult for static code analyzers to identify vulnerabilities in a precise manner. The main problem is with the typically high number of false positives reported by these tools, which refer to vulnerabilities that, in practice, do not exist. The common view is that the information regarding false positives is useless. In this paper we give an initial step towards investigating the hypothesis that false positives may be, in fact, a link to potential security problems. We analyzed 3 open-source web applications using a well-known static analyzer, then identified false positives and linked these to potential security problems. Preliminary results suggest that, in many cases, the presence of a false positive indicates a fragility of the application, which is prone, in different degrees, to turn into a real vulnerability.

Keywords—web applications; security; vulnerabilities; false positives; static analysis

I. INTRODUCTION

Web applications are becoming more and more popular day by day. In order to deliver advanced functionality they are also becoming more complex, which favors the emergence of new, hard to find, and potentially problematic vulnerabilities. The current time-to-market pressure combined with the typical lack of adequate preparation regarding security of applications, leads developers to overlook security in favor of the deployment of functionalities. This results in web applications that are exposed to huge numbers of users on the Internet, and that may carry serious security problems [1].

A valuable technique that can be used by programmers to improve the security of their applications is static code analysis. It consists in the analysis of computer software and is performed without having to run the program, unlike the dynamic techniques. This analysis is performed using the source code (or object code, depending on the tool used), and tries to detect vulnerabilities by searching for code patterns that are prone to be insecure [2].

The main and well-known problem arising from the use of static code analysis tools is the high number of false positives reported by these tools, which essentially are reports of vulnerabilities that do not exist [3]. This type of problem forces the user of such tools to manually review each potential vulnerability reported. This is considered to be a great loss of time because common opinion holds that false positives are

simple mistakes generated by the analyzer and, in general, a lower number of false positives suggest a more reliable, or at least more useful, tool.

In this paper we start to consider a different perspective on false positives and hypothesize that the indication of false positives by a static analyzer is not completely useless information and can, in fact, provide valuable information for developing more secure applications. A false positive is many times linked to poor programming practices; however, the particular code point pointed out by the analysis might hold potential for a real problem to occur. A real vulnerability might appear if the environment changes, if the code changes at another point of the program, or if there is another vulnerability that an attacker can explore (e.g., gaining privileged access to the file system or to the virtual machine being used). The view that we hypothesize has been signaled in previous industry reports [4], but concrete results on which type of information can be extracted and how it can help development is something that, to the best of our knowledge, has been largely disregarded in previous work.

To obtain a preliminary view on the importance and the type of information that can be extracted from false positives we designed an approach based on the selection of a few open-source Java web applications for managing web-based forums (Yazd Discussion Forum 3.0 [5], Apache Roller 5.1 [6] and mvnForum 2.1 [7]) and one well known static code analyzer (FindBugs 3.0.1 [3]). We then executed the analysis and used expert knowledge to identify false positives (in this initial study we target SQL Injection vulnerabilities). We grouped the false positives in a few different cases and tried to link each case to a real vulnerability created whenever the code or environment conditions change. These two situations (i.e., changes in the surrounding environment or changes in the code) that might generate new vulnerabilities are known to occur quite frequently in the nowadays very dynamic web environment.

The results obtained during these preliminary experiments suggest that false positives are often generated by pieces of code that match the code pattern that the static code analyzer is looking for, but, due to very small details, are not real vulnerabilities. In fact, in some cases, the presence of false positives indicates a fragility of the application that could easily lead to a presence of real vulnerabilities, whenever a specific change occurs. This type of information has some value that should not be discarded by developers, but could instead be used

to improve their development skills and build also more secure applications.

The outline of this paper is as follows. Section II presents background on static analysis and Section III presents the approach designed to understand the importance of false positives. Section IV presents the experimental evaluation, highlighting the links between false positives and potential security problems. Finally, Section V concludes the paper.

II. BACKGROUND

Static code analysis is a white-box technique that consists of analyzing the code of a particular application, without executing it [8]. Some static analyzer tools need source-code access, while for others, other forms of code (e.g., bytecode) are sufficient. This well-known and easy to use technique is many times used by developers to disclose bugs, to reveal bad code development practices, and also to uncover security problems in applications [9]. In practice, the technique is based on the analysis of the code trying match sections of the code with predefined patterns that represent problems. These problems are obviously dependent on expert knowledge, which is used precisely to define those patterns. This implies that analyzers might miss particular types of bugs, if the knowledge is incomplete [4].

Conceptually, the patterns used by static code analyzers can be loose or tight. A tight pattern is very precise, in the sense that it accurately matches a particular type of bugs, but in consequence may not be able to handle small changes in the patterns, which still correspond to a bug. On the other hand, a loose pattern allows finding unknown bugs better, but easily identifies bugs in the code, where there is no actual bug (i.e., it is a false positive).

In our context, we target specific types of bugs, which are named vulnerabilities and refer to internal faults that enable an external fault to harm the system [10]. Many static analyzers are able to find this type of patterns, and a false positive would be the report of a vulnerability regarding a code location where there is no vulnerability. As referred, a great disadvantage of using this type of technique is the typically high number of false positives generated by static analyzers. With the goal of mitigating this issue, in previous work authors have studied the conditions in the code that originate false positives [11]. Despite the existing work in the area, common opinion is that false positives bring in useless information, in this paper we try to understand if there can be some value in a report of a (security-related) false positive.

III. APPROACH

To understand the importance of false positives brought in by static analyzers regarding web application security, we designed an approach based on the following steps, which we explain in the next paragraphs.

- 1) Selection of one (or more) web application to be tested;
- 2) Selection of one (or more) static analysis tool, which will be used to test the web application selected in the previous step;
- 3) Execution of static analysis using the chosen testing tool against the selected web application;

- 4) Analysis of the tests results, using expert knowledge to identify false positives;
- 5) Using expert knowledge to identify links from false positives to real vulnerabilities.

In practice, we obviously need a system to be tested that provides us with the valuable information concerning false positives. In this paper, the target is set on web applications and, therefore, any application of this type will do, as long as we have access to the source code. In many cases, having byte code access will serve the static analyzers, but in the end we need to confirm all the vulnerabilities reported, and most of the time we can only do that by using the source-code.

The second step is to select one or more static analyzers that can be used to test the applications selected and produce a security report. If using more than one static code analyzer, they should be able to analyze the same vulnerability types being considered. For instance, in the context of the experiments in this paper we are targeting SQL Injection vulnerabilities, so if using several static code analyzers, they should be able to analyze this type of vulnerability.

Step three refers to the execution of the selected tools over the web applications, which must result in one report that points out the vulnerabilities identified. In the case of having executed multiple tools, the union of the vulnerabilities reported should be considered for analysis in step four. Here, an expert, or a team of experts, should analyze the vulnerabilities reported by the tools and agree on the identification of any possible false positives (vulnerabilities erroneously reported by the tools).

The final step involves resorting to expert knowledge to understand if and how could a false positive be directly transformed into a real vulnerability, without changing the purpose of the code (e.g., adding more functionality or removing current functionality). The goal is to understand if and how close a false positive is to becoming a real vulnerability. If this kind of link exists, false positive information can provide the developers with useful information regarding, at least, the quality of the code and the potential it has to become insecure.

IV. EXPERIMENTAL EVALUATION

This section presents the experiments conducted to evaluate the usefulness of information carried by false positives. We designed the experiments essentially to obtain very preliminary indicators regarding the following main questions presented in the next paragraphs:

- Which can be the reasons for the static code analyzer to report a piece of code as a vulnerability, although it is not?
- Can (and how) the reported false positives turn into real vulnerabilities?
- Can different false-positives be at different distances from becoming real vulnerabilities? What exactly can those distances be?
- Can the false positives information provide any indication regarding the security of the applications? If so, which type of indications can they provide?

It is important to emphasize that we do not aim to provide complete answers at the moment. The approach and experiments designed do not allow us to obtain such information in a complete manner. Currently, we are interested in obtaining valuable preliminary information that we can use to extend the approach in subsequent work. In the next subsections we present the experimental setup, the specific procedure carried out during the experiments, and discuss the results obtained.

A. Experiments configuration

In our experiments we selected three open source web applications to be analyzed. All applications serve the same purpose, which is to create and manage web forums, and are the following: i) Yazd Discussion Forum version 3.0 developed by Yasna Inc. [5]; ii) Apache Roller version 5.1 originally developed by Dave Johnson [6]; iii) mvnForum version 2.1 developed by MyVietnam [7].

The static code analyzer used in our experiments is FindBugs version 3.0.1, developed by the University of Maryland [3]. It analyzes Java programs, operating on their bytecode. Potential vulnerabilities are ranked from 1 to 20, and grouped into four categories: from "scariest" to "of concern", providing a hint to the user about their possible severity. FindBugs can be run from the command line, Ant, Maven, or in a standalone GUI application. There are also plug-ins available for popular Integrated Development Environments.

FindBugs is based on code pattern detectors and it is possible to add new ones using plug-ins. In order to have a more complete set of rules regarding security, we used the plugin Find Security Bugs version 1.4.4 [12]. FindBugs allows the use of filter files, which are XML documents used to include or exclude bug reports for particular type of vulnerabilities, classes and methods. As we focused on looking for false positives related to SQL injection vulnerabilities, we wrote a filter to exclude all bug reports not related with this type of vulnerability and then executed the tool against the selected web applications.

We analyzed FindBugs reports for each of the web applications, which implied manually checking each bug reported in order to understand if it is a real vulnerability or a false positive. For this purpose, we involved a total of four security researchers, all with experience in development. The researchers were required to analyze the tool report and perform a code review, to identify possible false positives. The final step involved identifying ways of transforming the false positives into real vulnerabilities, understanding how close they are to becoming a real problem. All researchers agreed on the outcome of the analysis process.

B. Results and Discussion

Table I presents an overview of the results obtained during the experiments. In particular, the table shows the web application tested, the number of SQL Injection vulnerabilities reported by FindBugs, the number of false positives found and also the different types of SQL Injection vulnerabilities identified by the researchers.

In the case of this application, FindBugs revealed a typical behavior of static analysis tools – a high number of false positives. Actually, all vulnerabilities reported by the tool were found to be false positives. Also, we found that the

TABLE I. OVERALL RESULTS

Web Application	Vulnerabilities reported Findbugs	False Positives	Types of vulnerabilities
Yazd 3.0	18	18	A, B
mvnForum 2.1	25	25	C, D
Apache Roller 5.1	3	3	D, E

vulnerabilities reported fall into the following five major cases, which we distinguished based on the functionality involved:

Case A: Use of a properties file that is read and concatenated with part of a query.

Case B: The use of a custom JDBC API Wrapper.

Case C: The creation of a query using concatenation of *final static* strings.

Case D: Concatenation of strings with values coming from the user, but the values are checked before they are used.

Case E: The execution of a complete SQL command which is read from a file.

As mentioned, all of the cases identified represent false positives. Case A resorts to a properties file to load part of an SQL command which is then concatenated with an application string and is correct, as the commands involved are secure. However, if an attacker has partial access to the machine and is able to tamper with the properties it may gain access to privileged information in the database. Note that, after gaining access to the machine, the attacker may not be able to directly access the database information due to the lack of access privileges, or because it may simply reside on another machine. In this case, being able to change the properties is the tool that the attacker needs. The same happens with Case E, which reads instead the complete SQL command from a text file but can turn into a real vulnerability in the case there is a non-authorized access to the machine and in particular to the text file used by the application.

In case B, the use of a custom wrapper over the SQL API represents in fact no security issue. Also, the group of researchers found no way of directly turning this false positive into a real vulnerability (without major source code changes).

In case C, a query is created using values that reside in a few classes in *final* attributes of type String. Although in some cases it is possible for a malicious class to use reflection to access, bypass visibility restrictions, bypass modifiers and actually change the values, this requires the ability to deploy and load the malicious class from a location where it has visibility to the class. There should be no security manager preventing such action and the server should allow hot deployments, so that the web application could be reloaded and include the malicious file. If it is not possible to perform a hot deployment, we should have the ability to change the server configuration and be allowed to restart the server. This shows that in this case, it can be much more difficult to obtain all the necessary conditions and suggests that this type of case is farther from turning into a real vulnerability, possibly requiring much less attention from developers.

Finally, in case D parts of an SQL command that are set by the user are concatenated with the rest of the SQL command that

resides in local method variables. Although this is apparently a real vulnerability, the parts that come from the user are in fact validated before being used in the code. This turns the insecure code into secure, however if the validation is not done correctly (i.e., if it does not check all possible cases) a security problem may arise. Thus, the link to a future vulnerability is on the possibly incomplete validation created by the developers, which essentially means that that particular part of the code is of critical importance regarding security. In Table II we summarize our findings, with respect to the distance to becoming a vulnerability (low, medium, high) per each of the five cases.

TABLE II. DISTANCE TO VULNERABILITY

Case	Distance
A	Low
B	–
C	–
D	Medium
E	Low

As we can see in Table II, we can separate the cases in different groups. Case B cannot be turned into any problem known by the researchers involved in the tests and case C would require an extremely unlikely combination of circumstances, especially the ability to change the deployment. Thus, we opted to not mark cases C and B, as they are quite different from the remaining. Despite this, both cases might alert the developer for bad practices or questionable design (e.g., generally there must be a strong motivation for creating a custom wrapper over something as popular as the JDBC API).

Cases A, D, and E are actually at a relatively short distance from turning into vulnerabilities and require attention from developers. However, while in cases A and E a small detail outside the application could enable the issue to turn into a vulnerability, for case D to turn into a vulnerability, a code change would have to occur. These are the reasons for marking A and E with *Low* and D with *Medium*. Having a systematic way to automatically identify and categorize this kind of results would greatly benefit developers helping them in creating more secure applications.

V. CONCLUSION

In this paper we presented preliminary work to investigate the hypothesis that false positives generated by static code analyzers are not useless, but might bring in useful information regarding the security of web applications. We carried out a set of experiments where we analyzed three Java web applications using the open source static analyzer FindBugs. We identified the false positives from the vulnerabilities reported by the tool and linked that information to potential vulnerabilities. Results suggest that the presence of a false positive is often due to a bad programming practice and small details might prevent it from becoming a real vulnerability. In the dynamic web environment, this possibility can easily turn into a problematic reality.

The different cases of false-positives found during the experiments show that sometimes there is no practical way of turning a false positive to a real vulnerability. They also show that some false positives are close to becoming a vulnerability, whereas others are quite far and require a very unlikely

combination of factors (and great attacker expertise) to be explored. Some cases also indicate that some parts of the code are critical in terms of security, as they are the last barrier (sometimes the only one) that prevent some part of the code from turning into a vulnerability.

Given the results observed, and despite being a short experiment, we believe it is expectable that developers can benefit from the information regarding the false positives (which they usually discard) for the development of web applications that are less prone to hold security problems. Future work includes the analysis of web applications using more static code analyzers, many more applications, and also analyzing different types of vulnerabilities, such as cross-site scripting (XSS). With a sufficiently large knowledge base we expect to be able to extract information that we can categorize and use to help developers creating more secure web applications.

ACKNOWLEDGMENTS

This work has been partially supported by the project DDesign, Verification and Validation of large-scale, dynamic Service SysEmS (DEVASSES), Marie Curie International Research Staff Exchange Scheme (IRSES) number 612569, within the context of the EU Seventh Framework Programme (FP7), by the project EUBra-BIGSEA, funded by the European Commission under the Cooperation Programme, Horizon 2020 grant agreement n° 690116, and by the European Commission Erasmus+ Traineeship program, Project N° 38629/15.

REFERENCES

- [1] P. J. C. Nunes, J. Fonseca, and M. Vieira, "phpSAFE: A Security Analysis Tool for OOP Web Application Plugins," in *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2015, pp. 299–306.
- [2] B. Chess and J. West, *Secure Programming with Static Analysis*. Pearson Education, 2007.
- [3] D. Hovemeyer and W. Pugh, "Finding bugs is easy," *ACM SIGPLAN Notices*, vol. 39, no. 12, pp. 92–106, 2004.
- [4] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World," *Commun. ACM*, vol. 53, no. 2, pp. 66–75, Feb. 2010.
- [5] Yasna Inc., "Yazd Discussion Forum Software," 2016. [Online]. Available: <http://www.forumsoftware.ca/>. [Accessed: 06-Apr-2016]
- [6] Apache Software Foundation, "Apache Roller," 2016. [Online]. Available: <https://roller.apache.org/>. [Accessed: 06-Apr-2016]
- [7] MyVietnam, "mvnForum," 2016. [Online]. Available: <https://sourceforge.net/projects/mvnforum/>. [Accessed: 06-Apr-2016]
- [8] G. J. Myers, *The art of software testing, Second Edition*. John Wiley & Sons Inc. New York, 2004.
- [9] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in java applications with static analysis," in *Proceedings of the 14th conference on USENIX Security Symposium - Volume 14*, Baltimore, MD, 2005, pp. 18–18.
- [10] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic Concepts and Taxonomy of Dependable and Secure Computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.
- [11] I. Medeiros, N. Neves, and M. Correia, "Detecting and Removing Web Application Vulnerabilities with Static Analysis and Data Mining," *IEEE Transactions on Reliability*, vol. 65, no. 1, pp. 54–69, Mar. 2016.
- [12] P. Arteau, "Find Security Bugs," 2016. [Online]. Available: <http://find-sec-bugs.github.io/>. [Accessed: 06-Apr-2016]